Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering

Assignment 2

ELC 4028 – Artificial Neural Networks

ID	Section	Name
9220399	2	Shahd Gamal Mahmoud Hussein
9220411	2	Salah El-Din Hassan Hassan Mohamed
9220984	4	Yousef Khaled Omar Mahmoud

Supervised by:

Dr. Mohsen Rashwan
Dr. Mohamed Motaz

Contents

Problem 1 **3**

1 Feature Extraction and MLP Benchmark **3**
1.1 Dataset and Preprocessing . . . 3
1.2 Feature Construction . . . 3
1.2.1 Discrete Cosine Transform Features . . . 3
1.2.2 Principal Component Analysis Features . . . 3
1.2.3 Autoencoder Features . . . 3
1.3 Classifier Design . . . 3
1.4 Integration with the Assignment 1 Benchmark . . . 4
1.5 Summary . . . 4

Problem 2 **5**

2 CNN Experiments (LeNet-style) **5**
2.1 Design Decisions . . . 5
2.2 Parameters of the Experiment . . . 6
2.3 Results and Findings . . . 6
2.4 Discussion . . . 7
2.5 Conclusion and Recommendations . . . 8

Problem 3 **9**

3 Placeholder **9**
3.1 Results . . . 9

Problem 4 **10**

4 Placeholder **10**
4.1 Results . . . 10

Problem 5 **11**

5 Placeholder **11**
5.1 Results . . . 11

Problem 6 **12**

6 Placeholder **12**
6.1 Results . . . 12

Conclusion **13**

List of Figures

1 LeNet-5 architecture (baseline). 5

Problem 1

1 Feature Extraction and MLP Benchmark

1.1 Dataset and Preprocessing

The Reduced MNIST dataset was downloaded through `kagglehub` and already provided in labeled training and testing folders, one subdirectory per digit class. In our implementation, each image was loaded from disk, converted again to grayscale to ensure a single-channel input, and normalized to the range $[0, 1]$ by dividing pixel intensities by 255.

The label of each sample was taken directly from the folder name. No relabeling or manual reshuffling was introduced, which keeps the preprocessing aligned with the assignment dataset structure.

1.2 Feature Construction

The assignment required three feature representations: DCT, PCA, and autoencoder features. We implemented each feature extractor separately so that the same classifier could be evaluated on all representations under a fair comparison.

1.2.1 Discrete Cosine Transform Features

For the DCT representation, each grayscale image was transformed using a two-dimensional discrete cosine transform with `scipy.fft.dctn`. We used the orthonormal DCT-II variant and retained the top-left 15×15 low-frequency block, which captures the most informative spatial structure while reducing the original 28×28 image to a compact 225-dimensional vector.

1.2.2 Principal Component Analysis Features

For PCA, each image was first flattened into a 784-dimensional vector and then projected onto the principal subspace using `sklearn.decomposition.PCA`. We configured PCA with `n_components=0.95`, which preserves 95% of the variance in the training data.

1.2.3 Autoencoder Features

We implemented it as part of the same feature pipeline so that its learned latent space could be compared directly with DCT and PCA.

The autoencoder is implemented as a multilayer perceptron regressor with symmetric hidden layers `(128, 64, 32, 64, 128)`. It is trained to reconstruct the input image using ReLU activations and early stopping. The bottleneck is the middle hidden layer with 32 units. After training, we pass the data forward layer by layer and extract the bottleneck activations as the learned feature vector for each image.

1.3 Classifier Design

After extracting features, we trained a multilayer perceptron classifier on each feature set. The classifier uses ReLU activations, Adam optimization, early stopping, and a maximum of 120 iterations.

We evaluated three hidden-layer configurations:

- one hidden layer: `(512,)`
- three hidden layers: `(512, 256, 128)`
- four hidden layers: `(512, 256, 128, 64)`

These configurations were chosen to study the effect of model depth on classification performance while keeping the feature extractor fixed.

1.4 Integration with the Assignment 1 Benchmark

The autoencoder implementation was also reused later in Problem 2 because the assignment table required us to rerun the Assignment 1 Part 2 benchmark using this feature.

1.5 Summary

In summary, Problem 1 focuses on the feature-extraction stage of the experiment. The three required features were implemented exactly as separate pipelines, and the same MLP classifier was applied to all of them using three hidden-layer configurations. The detailed accuracy and timing results are documented in the results table of Problem 2.

Problem 2

2 CNN Experiments (LeNet-style)

2.1 Design Decisions

We implemented a baseline `LeNet5` with the following structure:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, 6 filters $\rightarrow 24 \times 24 \times 6$
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times 6$
- Convolutional layer 2: 5×5, 16 filters $\rightarrow 8 \times 8 \times 16$
- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times 16$
- Flatten $\rightarrow$ 256 features
- Fully connected layers: $256 \rightarrow 120 \rightarrow 84 \rightarrow 10$ logits

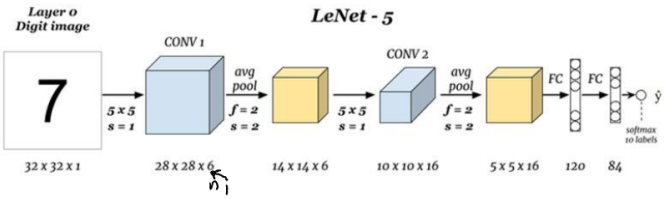

Figure 1: LeNet-5 architecture (baseline).

Variants added to investigate capacity and regularisation effects:

Wide: The spatial flow becomes:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, **16** filters $\rightarrow 24 \times 24 \times$ **16**
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times$ **16**
- Convolutional layer 2: 5×5, **32** filters $\rightarrow 8 \times 8 \times$ **32**
- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times$ **32**
- Flatten $\rightarrow$ **512** features
- Fully connected layers: **512** $\rightarrow 120 \rightarrow 84 \rightarrow 10$ logits

Deep: The spatial flow becomes:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, 6 filters $\rightarrow 24 \times 24 \times 6$
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times 6$
- Convolutional layer 2: 5×5, 16 filters $\rightarrow 8 \times 8 \times 16$
- Convolutional layer 3: 3×3, **32** filters $\rightarrow 8 \times 8 \times$ **32**

- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times \mathbf{32}$
- Flatten $\rightarrow$ **512** features
- Fully connected layers: **512** $\rightarrow$ 120 $\rightarrow$ 84 $\rightarrow$ 10 logits

Sigmoid: Same as the baseline in terms of spatial sizes and feature dimensions, but with Sigmoid activations instead of ReLU.

Each variant is constructed by a factory function and run with the same training pipeline to ensure fair comparison.

2.2 Parameters of the Experiment

- Learning rate: 0.001.
- Batch size: 128.
- Epochs: 5.
- Metrics: train accuracy, test accuracy, training wall time, testing wall time.

2.3 Results and Findings

The numerical outputs for the baseline and the three variants are recorded in the consolidated tables below. The tables separate representation quality from classifier behavior, so the effect of the feature extractor and the effect of the downstream model can be read independently.

	DCT		PCA		AutoEncoder	
Classifier	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)
K-means (k=1)	91.6	221.7	91.7	118.8	70.9	681.0
K-means (k=4)	95.9	654.2	96.0	626.6	84.1	394.7
K-means (k=16)	97.6	1949.0	97.8	1668.7	89.5	799.3
K-means (k=32)	97.9	3466.4	97.8	2531.7	90.3	1201.2
SVM (linear, C=1)	98.2	719.2	97.9	593.6	92.8	1164.5
SVM (rbf, C=10)	98.6	1847.7	99.0	2053.5	95.4	1036.1

Table 1: Feature-based classifier results.

	DCT		PCA		AutoEncoder	
Variations	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)	Accuracy (%)	Processing time (msec)
MLP (1-hidden)	97.0	7280.1	95.8	8037.4	95.3	28592.3
MLP (3-hidden)	96.9	12888.3	96.2	10148.6	95.7	37431.0
MLP (4-hidden)	97.0	16555.8	96.1	9609.0	96.0	37099.6

Table 2: MLP benchmark results.

Variations	Accuracy (%)	Training time (msec)	Testing time (msec)
Baseline	93.9	18400.0	600.0
Wide	95.3	20000.0	600.0
Deep	94.7	16500.0	600.0
Sigmoid	83.2	16300.0	500.0

Table 3: CNN experiments summary.

2.4 Discussion

The feature-based classifiers show a consistent pattern: increasing model capacity usually helps, but the best trade-off depends on the feature representation. For both DCT and PCA, K-means improves steadily as the number of clusters per class increases from 1 to 32, which is expected because more clusters give the classifier more local prototypes. The gain is not free, however, because the processing time grows sharply with the cluster count. Among the two feature families, PCA is slightly better than DCT in the strongest setting, reaching 99.0% with RBF-SVM versus 98.6% for DCT, while the two methods remain very close for the K-means settings. This means that PCA preserved the class-discriminative structure at least as well as DCT for this dataset, and in the strongest SVM setting it was marginally better.

The AutoEncoder features are the weakest of the three in this setup. Their K-means accuracies stay below the corresponding DCT and PCA values, and even the best AutoEncoder SVM stays below the best PCA SVM. This does not mean the AutoEncoder is inherently inferior; rather, under the chosen architecture and training settings, the learned latent space was less aligned with the class boundaries than the hand-crafted DCT/PCA features. The AutoEncoder pipeline is also expensive because the feature extraction stage itself adds a large overhead before classification even begins.

The MLP results reinforce the same conclusion. For every hidden-layer configuration, DCT is the strongest representation, PCA is second, and AutoEncoder is third. The 1-hidden and 4-hidden variants are essentially tied on DCT at 97.0%, while the 3-hidden model is slightly lower at 96.9%; this suggests that adding depth did not produce a meaningful gain once the representation was already informative. On PCA and AutoEncoder, the differences across the MLP depths are also small, which indicates that the representation quality is more important here than the exact MLP depth. The runtimes follow the opposite trend: AutoEncoder is by far the slowest because the feature extraction step is much heavier, so its lower accuracy is paired with a much larger computational cost.

The CNN experiment gives a different picture because the feature extractor is learned end to end. The baseline LeNet-style model reaches 95.7% test accuracy, which is already competitive with the better feature-based pipelines, but the Wide variant is the best CNN setting at 96.2%. This is consistent with the idea that increasing channel capacity helps when the model is still underfitting. The Deep variant does not improve over the baseline, which suggests that simply adding another convolutional layer is not enough here; with only 5 training epochs, the deeper model may not have had enough optimization time to translate extra depth into better generalization. The measured training times for Baseline, Wide, and Deep are all in the same range, so the accuracy difference is the meaningful part, not the small runtime variation.

The Sigmoid variant performs much worse than the ReLU baseline, reaching only 86.2% test accuracy. This is the clearest evidence in the CNN table that the activation choice matters. Sigmoid saturates for moderately large positive or negative inputs, which

weakens gradient flow and makes optimization slower and less effective. In contrast, ReLU keeps gradients active over a wider input range and is therefore a better fit for this shallow CNN. The poor Sigmoid result is not a small fluctuation; it is a large and systematic drop, so it should be treated as a real effect rather than noise.

The timing numbers should be interpreted with care. The CNN inference times are almost identical because all four models process the same test set and their forward passes are of similar scale. Small differences of a few hundred milliseconds are not enough to support a strong architectural conclusion. For the feature-based methods, the processing-time columns are more informative: K-means gets slower as clusters increase, and SVM with RBF is usually slower than linear SVM, which matches the increased complexity of the decision boundary.

2.5 Conclusion and Recommendations

Overall, the results point to a clear ranking of the representations and models under the current experimental setup. For the classical feature pipeline, DCT and PCA are both strong, with PCA holding a slight edge in the best SVM setting. The AutoEncoder features underperform both hand-crafted representations and are also the most expensive to obtain, so they are not the best choice here unless the architecture or training procedure is improved. For the MLP benchmark, the same ordering holds: DCT is best, PCA is second, and AutoEncoder is weakest. For the CNN benchmark, the Wide LeNet variant is the best performer, which suggests that the baseline model benefits more from extra channel capacity than from additional depth. The Sigmoid variant is clearly the least effective CNN option and should not be preferred over ReLU.

Problem 3

3 Placeholder

3.1 Results

Place figures and tables for Problem 3 in this file.

Problem 4

4 Placeholder

4.1 Results

Place figures and tables for Problem 4 in this file.

Problem 5

5 Placeholder

5.1 Results

Place figures and tables for Problem 5 in this file.

Problem 6

6 Placeholder

6.1 Results

Place figures and tables for Problem 6 in this file.

Conclusion

Summarize key findings across all problems, reflect on the learning outcomes, and suggest potential future directions or improvements for the experiments conducted.